HYPERSONICS / CFD

Scramjet intake design at Mach 6

Single-ramp intake design, inviscid validation and a RANS comparison at ISA 20 km freestream conditions.

Toran Bahadur Niroula

Department of Mechanical and Aerospace Engineering
Pulchowk Campus, IOE, Tribhuvan University
April 2026

In this report

Freestream conditions and intake design

Inviscid CFD and performance comparison

RANS flow field and discussion

Python code: intake.py

Separated from the original Hypersonics and Advanced Propulsion report. Equations, result figures and technical text retain their original numbering. Shared setup material is included where needed. No simulation results have been changed.

Table 1: Freestream conditions (ISA 20 km, $\gamma = 1.4$, $C_p = 1004.5\,\mathrm{J/(kg\,K)}$).

Quantity	**Symbol**	**Value**	**Unit**
Static pressure	p_∞	5529	Pa
Density	ρ_∞	0.088	$\mathrm{kg/m^3}$
Temperature	T_∞	216.65	K
Speed of sound	a_∞	≈ 295	m/s
Dynamic viscosity	μ	$\approx 1.42 \times 10^{-5}$	Pa·s

Advanced Propulsion

1. Introduction

This part covers the oblique-shock based design and CFD validation of a single-ramp scramjet intake at $M_\infty = 6$ (ISA 20 km). An inviscid simulation confirms the design point performance targets, and a RANS simulation with k-ω SST quantifies viscous effects. Freestream conditions are as given in Table 1.

2. Intake Design

2.1. Design Requirements

Table 3: Scramjet intake design requirements.

Parameter	Symbol	Value
Flight Mach number	M_∞	6
Static pressure ratio	π	20
Isolator height	h_{iso}	50 mm

2.2. Oblique Shock Design Procedure

The intake is a single-ramp system. The ramp angle δ is found by simultaneously solving the oblique shock pressure ratio and the θ-β-M relation:

$$\frac{p_2}{p_1} = 1 + \frac{2\gamma}{\gamma+1}\left(M_\infty^2 \sin^2\beta - 1\right), \tag{5}$$

$$\tan\theta = \frac{2\cot\beta\left(M_\infty^2\sin^2\beta - 1\right)}{M_\infty^2(\gamma + \cos 2\beta) + 2}. \tag{6}$$

With $\pi = 20$, $M_\infty = 6$, $\gamma = 1.4$, the solution gives $\theta = 14.15°$ and $\beta = 23.38°$.

The total pressure recovery is:

$$\text{TPR} = \frac{p_{02}}{p_{01}} = \left[\frac{(\gamma+1)M_\infty^2\sin^2\beta}{(\gamma-1)M_\infty^2\sin^2\beta + 2}\right]^{\gamma/(\gamma-1)} \cdot \left[\frac{(\gamma+1)}{2\gamma M_\infty^2\sin^2\beta - (\gamma-1)}\right]^{1/(\gamma-1)}. \tag{7}$$

Ramp length and capture height follow from the shock impingement geometry:

$$L = \frac{h_{\text{iso}}}{\tan(\beta-\theta)}, \qquad H = h_{\text{iso}} + L\tan\theta. \tag{8}$$

Table 4: Intake geometry from oblique shock design.

Parameter	Symbol	Value
Ramp angle	θ	14.15°
LE Shock angle	β_1	21.77°
Cowl Shock angle	β_2	26.15°
Ramp length	L	1.026 m
Capture height	H	0.301 m
Isolator height	h_{iso}	50 mm
Design TPR		0.50
Design π		20

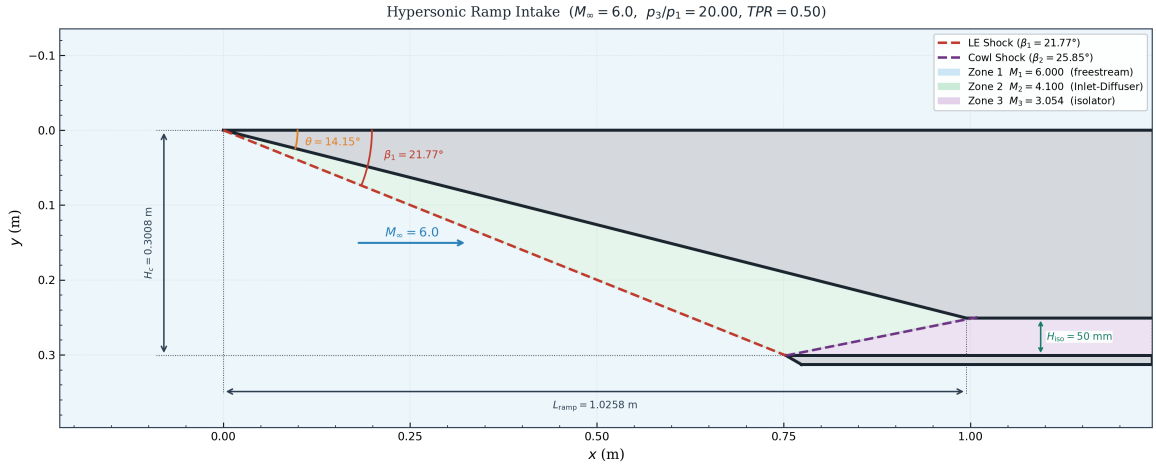


Figure 20: Scramjet ramp intake geometry.

3. CFD Setup

Both the inviscid and RANS simulations use `rhoCentralFoam` with Kurganov-Tadmor flux and `vanLeer` reconstruction. The RANS case adds the k-ω SST model. The structured mesh of 190,000 cells was generated with `blockMesh`. Refinement is applied near the ramp wall and in the oblique shock region.

Table 5: CFD simulation parameters.

Parameter	Inviscid	RANS
Solver	`rhoCentralFoam`	`rhoCentralFoam`
Turbulence	None	k-ω SST
Cell count	190,000	190,000
M_∞	6	6
Wall BC	Slip	No-slip, adiabatic
Flux scheme	Kurganov-Tadmor	
Δt	Adaptive ($\text{Co}_{\max} = 0.4$)	

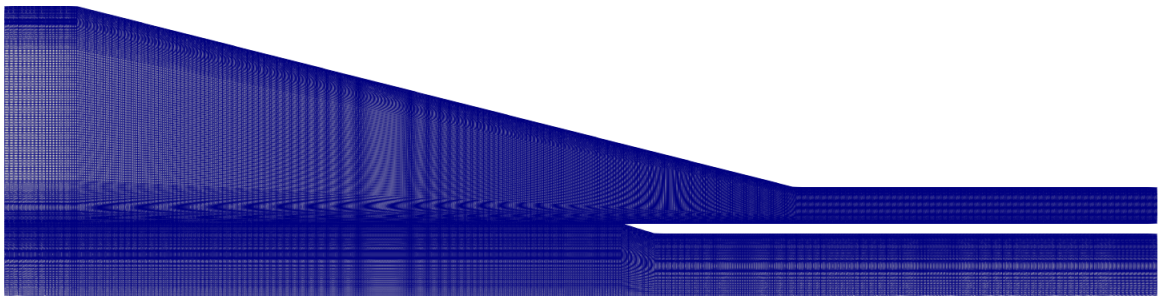


Figure 21: Computational mesh (190,000 cells).

4. Inviscid CFD Results

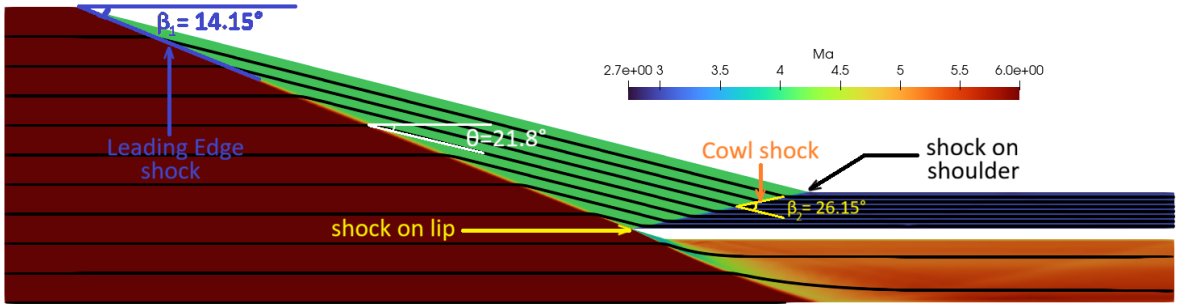
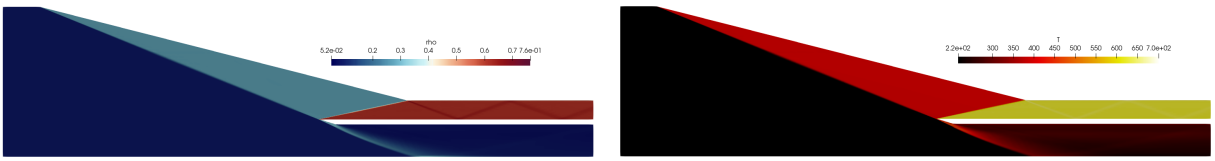


Figure 22: streamlines and flow features in single ramp intake.



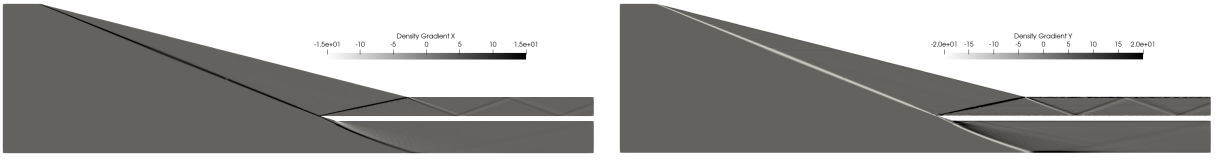
(a) Mach number. (b) Static pressure.

Figure 23: Inviscid flow field, $M_\infty = 6$.



(a) Density. (b) Temperature.

Figure 24: Inviscid density and temperature fields.



(a) $\partial\rho/\partial x$. (b) $\partial\rho/\partial y$.

Figure 25: Numerical Schlieren for the inviscid intake.

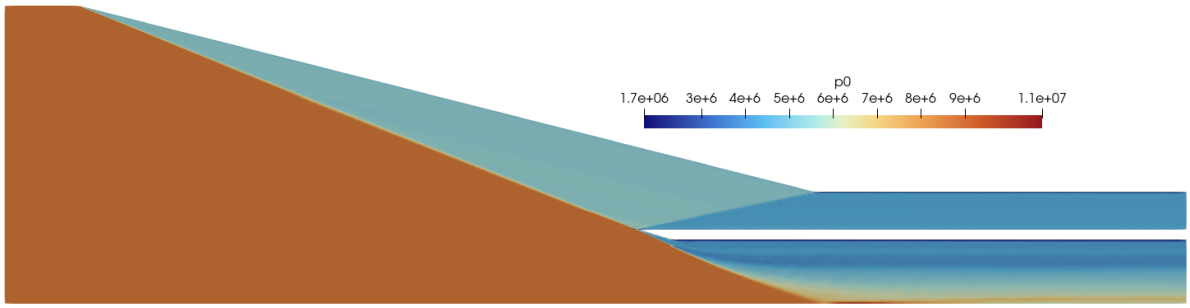


Figure 26: Total pressure field.

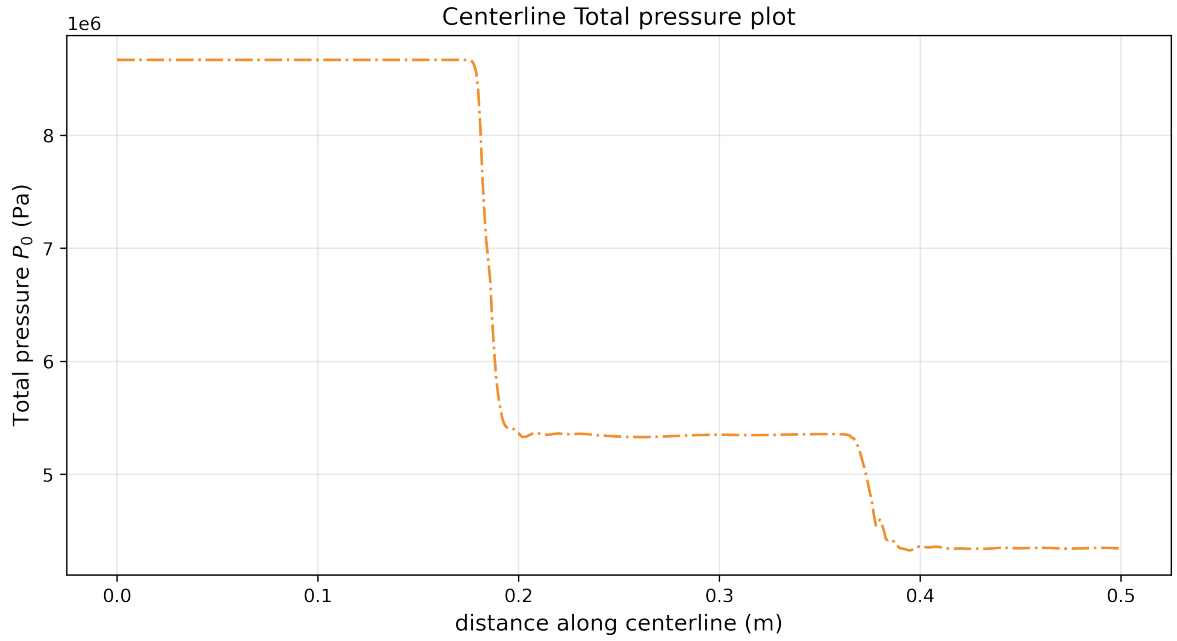


Figure 27: Centreline total pressure profile.

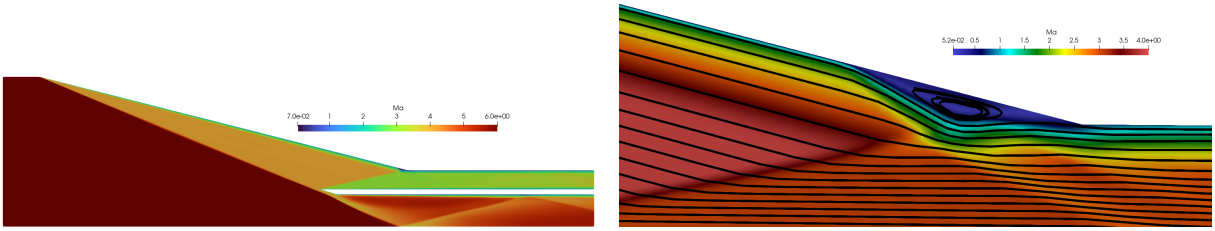
4.1. Performance Comparison

Table 6: Design vs. inviscid CFD performance at the isolator exit plane.

Parameter	Design	Inviscid CFD	Deviation
Total pressure recovery (TPR)	0.500	0.5013	+0.26%
Static pressure ratio π	20.00	19.928	−0.36%

Both parameters agree with design targets to within 0.4%. Since the inviscid CFD and design procedure share the same governing equations, the residual discrepancy reflects numerical dissipation in the Kurganov scheme and finite mesh resolution.

5. RANS CFD Results



(a) Mach number (RANS). (b) Streamlines (RANS).

Figure 28: RANS flow field.

Boundary layer thickening on the ramp surface displaces the effective flow area relative to the inviscid case, and the shock impingement region becomes more diffuse. The streamlines show a thickened boundary layer near the shoulder where the cowl shock impinges, and a small separation bubble is visible at that location. Because of the boundary layer growth, the effective deflection angle seen by the flow increases, which means the leading-edge shock no longer impinges exactly at the cowl as in the inviscid case.

The k-ω SST model is well suited to this problem. Its blending between the k-ω formulation near the wall and k-ε in the freestream handles the adverse pressure gradient at shock impingement. A known limitation is the fully turbulent assumption. If the boundary layer remains laminar over a significant portion of the ramp, the model will overestimate skin friction and underestimate any separation length.

6. Discussion

The less than 0.4% deviation between design and inviscid CFD confirms that the design procedure is internally consistent. Viscous effects (RANS) produce boundary-layer displacement that slightly reduces the effective compression area. The single-ramp TPR of 0.50 is characteristic of a strong oblique shock at Mach 6. A two or three ramp system would split the compression across weaker shocks and recover substantially more total pressure, at the cost of increased geometric complexity.

Listing 3: Scramjet intake oblique shock design (`intake.py`).

```
import math

GAMMA = 1.4
M1, PI, H_ISO = 6.0, 20.0, 0.05

def tbm(M, b):
    sb = math.sin(b)
    Mn1sq = (M * sb) ** 2
    return math.atan(2*(Mn1sq-1) / (math.tan(b)*(M**2*(GAMMA+math.cos(2*b))+2)))

def solve_beta(M, theta_deg):
    theta = math.radians(theta_deg)
    mu = math.asin(1/M)
    beta = mu + math.radians(3)
    for _ in range(300):
        f  = tbm(M, beta) - theta
        df = (tbm(M, beta+1e-7) - tbm(M, beta-1e-7)) / 2e-7
        if abs(df) < 1e-15: break
        beta = max(mu+1e-6, min(beta + max(min(-f/df, math.radians(0.5)), math.
            radians(-0.5)), math.pi/2-1e-6))
        if abs(f) < 1e-10: break
    return beta

def p_ratio(M, b):
    Mn1 = M * math.sin(b)
    return (2*GAMMA*Mn1**2 - (GAMMA-1)) / (GAMMA+1)

def rho_ratio(M, b):
    Mn1 = M * math.sin(b)
    return ((GAMMA+1)*Mn1**2) / ((GAMMA-1)*Mn1**2 + 2)

def M_down(M, b, theta):
    Mn1 = M * math.sin(b)
    Mn2sq = (1 + (GAMMA-1)/2*Mn1**2) / (GAMMA*Mn1**2 - (GAMMA-1)/2)
```

```
    return math.sqrt(Mn2sq) / math.sin(b - theta)

def max_deflection(M):
    mu = math.asin(1/M)
    return max(tbm(M, math.radians(b)) for b in range(int(math.degrees(mu))+1,
        90))

def solve_theta(M1, pi_target):
    lo, hi = 0.1, math.degrees(max_deflection(M1)) * 0.95
    for _ in range(120):
        theta = 0.5*(lo+hi)
        b1 = solve_beta(M1, theta)
        M2 = M_down(M1, b1, math.radians(theta))
        if theta >= math.degrees(max_deflection(M2)):
            hi = theta; continue
        b2 = solve_beta(M2, theta)
        p31 = p_ratio(M1, b1) * p_ratio(M2, b2)
        if abs(p31 - pi_target) < 1e-8: break
        lo, hi = (theta, hi) if p31 < pi_target else (lo, theta)
    return 0.5*(lo+hi)

theta_deg = solve_theta(M1, PI)
theta     = math.radians(theta_deg)
b1        = solve_beta(M1, theta_deg)
M2        = M_down(M1, b1, theta)
b2        = solve_beta(M2, theta_deg)
M3        = M_down(M2, b2, theta)
p21       = p_ratio(M1, b1)
p32       = p_ratio(M2, b2)
p31       = p21 * p32

m    = math.tan(theta - b2)
x_s  = H_ISO * (m - math.tan(b1)) / (m * (math.tan(b1) - math.tan(theta)))
L_ramp = x_s / math.cos(theta)

t1 = 1 + 0.5*(GAMMA-1)*M1**2
t3 = 1 + 0.5*(GAMMA-1)*M3**2
tpr = p31 * (t3/t1) ** (GAMMA/(GAMMA-1))

print(f"Inputs:  M={M1}  p3/p1={PI}  H_iso={H_ISO} m")
print(f"Theta (ramp angle):   {theta_deg:.4f} deg")
print(f"Beta1 (LE shock):     {math.degrees(b1):.4f} deg")
print(f"Beta2 (cowl shock):   {math.degrees(b2):.4f} deg")
print(f"Ramp length:          {L_ramp:.5f} m")
print(f"Total pressure recovery: {tpr:.5f}")
```